📋 Project Overview: ThesisGuard AI

ThesisGuard AI is an advanced Retrieval-Augmented Generation (RAG) web application and academic peer review suite built to assist students and researchers in preparing for university thesis defenses. It ingests research papers or data sheets, indexes them into a vector space, and answers rigorous academic questions with source citations.

PY+ 1

🛠️ Tech Stack & Dependencies

The project is built on a robust Python ecosystem specifically designed for vector search and LLM orchestration:

TXT

- **Frontend & UI**: streamlit for creating an interactive, wide-layout web dashboard with custom styling.

PY

- **LLM & Embeddings**: langchain, langchain-openai, langchain-core, and langchain-community for managing prompt templates, runnables, and OpenAI integrations.

PY

- **Vector Store & Chunking**: faiss-cpu for efficient similarity vector search, paired with langchain-text-splitters (RecursiveCharacterTextSplitter).

PY

- **Document Loaders**: pypdf for parsing PDF research papers and pandas / openpyxl for reading Excel metrics sheets.

PY

- **Utilities**: tiktoken for token-level text handling.

TXT

⚙️ Core Architecture & Code Breakdown (app.py)

1. Secure Access Portal (Lock Screen)

The application opens behind a secure password-masked authentication gate:

- Users input their OpenAI API key.

PY

- Clicking **Unlock Dashboard** validates the key instantly by initializing a test instance of OpenAIEmbeddings, storing credentials in st.session_state, and triggering st.rerun().

PY

2. Configurable Sidebar Controls

The sidebar provides deep customizability over the RAG pipeline:

PY

- **Document Toggle**: Choose between **PDF Research Paper** or **Excel Metrics Sheet** upload modes.

PY

- **Chunking Parameters**: Adjust **Chunk Size** (200–2000 tokens) and **Chunk Overlap** (0–300 tokens).

PY

- **Retrieval Settings**: Control top_k chunk retrieval depth (1–10) and select academic rigor levels (*Master's Defense*, *PhD Dissertation*, *Undergraduate Review*).

PY

3. FAISS Vector Indexing & Caching (@st.cache_resource)

- **PDF Processing**: Uses a temporary file writer and PyPDFLoader to extract raw text from research papers.

PY

- **Excel Processing**: Converts multi-sheet workbooks into structured Markdown/CSV summaries via pandas and wraps them into LangChain Document objects with metadata.

PY

- **Splitting & Embedding**: Splits documents recursively and embeds them into a local FAISS vector store using OpenAI embeddings.

PY

4. Strict Academic RAG Chain

- Employs a custom ChatPromptTemplate enforcing a strict academic reviewer persona matching the selected rigor level.

PY

- **Rules**: Answers strictly using provided context, outputs a standard fallback phrase if info is missing, and explicitly cites page or sheet numbers (e.g., (p. 3)).

PY

- Utilizes a modern LangChain Expression Language (LCEL) chain combining retriever formatting, prompt templating, LLM generation (gpt-4o-mini or gpt-4o), and a string output parser.

PY

5. Transparency & Source Auditing

- **Centered Chat Interface**: Features persistent message history replay and pinned bottom-center chat inputs.

PY

- **Retrieved Chunks Expander**: Displays a transparency drawer (st.expander) for every assistant response, showing exact chunk excerpts, source file names, and page/sheet numbers for auditability.

PY